Vector Compression for High-Dimensional Data

A survey of linear-algebra compression methods with an honest retrieval evaluation

Clayton Schmidter
MATH 5110, Applied Linear Algebra
github.com/claytomode/math5110-vector-compression

Abstract

Modern AI systems store billions of high-dimensional vectors (text embeddings, attention key/value caches, and retrieval indexes) whose footprint scales as $n \cdot d \cdot$(bits per dimension). This paper surveys four classical linear-algebra tools for shrinking such vectors (Johnson–Lindenstrauss random projection, truncated SVD, 1-bit sign quantization, and uniform scalar quantization) and a recent two-stage method, TurboQuant, that combines an MSE-optimal scalar quantizer with a 1-bit quantized-JL residual correction. We implement pedagogical versions of each method in NumPy and evaluate them not only on geometric distortion but on a realistic retrieval task: a 1,380-chunk retrieval-augmented-generation (RAG) index built from a linear-algebra textbook, queried by 300 auto-generated questions and scored against full-precision ground truth. Our central finding is that *distance preservation does not imply rank preservation*: Johnson–Lindenstrauss preserves pairwise geometry yet retains only 39% of the true top-3 neighbors, while a two-line scalar quantizer at 4 bits retains 93%. TurboQuant wins decisively at aggressive bit budgets (76% vs 63% top-3 overlap against scalar at matched stage-1 bits), but plain scalar quantization is the stronger choice once 4–8 bits per dimension are affordable. We argue that compression methods must be compared on *measured* bits per dimension and on a *retrieval* metric, not on nominal labels or distance error alone.

1 Introduction

A single OpenAI `text-embedding-3-small` vector at $d = 256$ dimensions costs $256 \times 32 = 8{,}192$ bits in `float32`. A modest corpus of one million chunks therefore costs roughly one gigabyte before any index structure is added, and production systems routinely store far more. The same arithmetic governs the attention key/value cache inside a running language model and the lookup table of token embeddings. In every case the cost is the product of three terms:

$$\text{storage} = n \cdot d \cdot (\text{bits per dimension}). \tag{1}$$

Reducing any factor without destroying the geometry that downstream tasks depend on is the core problem of **vector compression**. The relevant geometry is almost always an inner product: retrieval ranks candidates by cosine similarity, and attention scores keys by dot product. When embeddings are L2-normalized, cosine similarity equals the dot product, so a compressor that approximately preserves inner products approximately preserves both ranking and attention.

This project has three parts, moving from theory to computation to application:

- **Part 1 (Survey).** The linear algebra of four compression families plus the TurboQuant pipeline.

- **Part 2 (Computation).** NumPy implementations and two metric families: geometric distortion and retrieval overlap.
- **Part 3 (Application).** A compressed textbook RAG index with a 300-query automatic evaluation and a live search UI.

Contributions. (1) A unified, from-scratch implementation of five compression methods sharing one embedding matrix and random seed. (2) An honest retrieval benchmark that uses full-precision top-k as ground truth rather than synthetic labels. (3) A clear empirical separation between *geometry-preserving* and *rank-preserving* methods, with practical guidance on when each method wins.

2 Background and theory

Throughout, vectors live in $\mathbb{R}^d$. A query q is scored against keys $k_1, \ldots, k_n$ by cosine similarity (equivalently, dot product after normalization). A compressor replaces each key with a cheaper representation whose scores $\hat{s}(q, k_i)$ approximate the true scores $s(q, k_i)$.

2.1 Johnson–Lindenstrauss random projection

The Johnson–Lindenstrauss (JL) lemma states that n points in $\mathbb{R}^d$ can be linearly embedded into $\mathbb{R}^k$ with all pairwise distances preserved up to a factor of $(1 \pm \varepsilon)$, provided

$$k = O\big(\varepsilon^{-2} \log n\big)\,. \tag{2}$$

Critically, k depends on the number of points and the tolerance, not on the ambient dimension d. We use the standard Gaussian construction: a random matrix $R \in \mathbb{R}^{k \times d}$ with independent entries $R_{ij} \sim \mathcal{N}(0, 1/k)$, applied as a sketch $y = Rx \in \mathbb{R}^k$.

Why distances are preserved. Fix a vector x and examine the squared sketch length. Each coordinate $y_i = \sum_j R_{ij} x_j$ is Gaussian with mean 0 and variance $\|x\|^2/k$, so

$$\mathbb{E}\,\|Rx\|^2 = \sum_{i=1}^{k} \mathbb{E}\, y_i^2 = k \cdot \frac{\|x\|^2}{k} = \|x\|^2, \tag{3}$$

which makes the projection an isometry in expectation. The quantity $\|Rx\|^2$ is an average of k independent squared Gaussians, a χ^2 random variable, and a standard concentration bound gives

$$\Pr\big[\,\big|\,\|Rx\|^2 - \|x\|^2\,\big| \geq \varepsilon \|x\|^2\,\big] \leq 2\exp\!\left(-\frac{k}{4}\big(\varepsilon^2 - \varepsilon^3\big)\right). \tag{4}$$

Applying this to each of the $\binom{n}{2}$ difference vectors $x_i - x_j$ and taking a union bound, the failure probability is below 1 as soon as $k = O(\varepsilon^{-2} \log n)$, which is the lemma. Inner products follow from the polarization identity

$$\langle x, y \rangle = \tfrac{1}{4}\big(\|x + y\|^2 - \|x - y\|^2\big); \tag{5}$$

since the sketch preserves every norm on the right-hand side, it preserves the dot product on the left, and therefore cosine similarity, in expectation.

Why ranking still breaks. Each output coordinate is a random mixture of all d inputs, drawn before any data is seen, so JL is a *data-oblivious* compressor: the required k depends only on n and ε, never on d. Averaging k such mixtures concentrates the sketch length, but the residual error keeps a fixed relative scale of order ε. Top-k retrieval depends on the *order* of distances, not their size, and when the true nearest neighbors are separated by gaps smaller than this ε noise floor the projection reorders them. In our setting $d = 256$, $n = 1{,}380$, and $\varepsilon = 0.3$ give $k \approx 128$ (a nominal $2\times$ compression); the lemma then guarantees the magnitude of pairwise distances at that tolerance while promising nothing about their order. This is the gap between **distance preservation and rank preservation** that our experiments expose.

2.2 Truncated (rank-k) SVD

The singular value decomposition factors the stacked embedding matrix $A \in \mathbb{R}^{n\times d}$ as $A = U\Sigma V^\top$, with orthonormal U and V and singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$ on the diagonal of Σ. Truncating to the top k directions gives

$$A_k = U_k \Sigma_k V_k^\top = \sum_{i=1}^{k} \sigma_i \, u_i v_i^\top. \tag{6}$$

The right singular vectors $V_k \in \mathbb{R}^{d\times k}$ span the k-dimensional subspace of maximum variance across the corpus. Each vector is compressed by projecting into that subspace and reconstructed by mapping back:

$$z = V_k^\top x \in \mathbb{R}^k, \qquad \hat{x} = V_k V_k^\top x. \tag{7}$$

The operator $P_k = V_k V_k^\top$ is an orthogonal projector, satisfying $P_k^2 = P_k = P_k^\top$, so the residual $x - \hat{x}$ is orthogonal to the retained subspace and the Pythagorean relation $\|x\|^2 = \|\hat{x}\|^2 + \|x - \hat{x}\|^2$ holds exactly.

Optimality and exact error. Where JL is data-oblivious, SVD is data-optimal: it spends its k dimensions on exactly the directions in which the corpus varies most. By the Eckart–Young theorem, A_k is the best rank-k approximation of A in both the Frobenius and spectral norms, with error fixed precisely by the discarded singular values,

$$\|A - A_k\|_F^2 = \sum_{i>k} \sigma_i^2, \qquad \|A - A_k\|_2 = \sigma_{k+1}. \tag{8}$$

The fraction of energy (variance) retained is $\sum_{i\leq k} \sigma_i^2 / \sum_i \sigma_i^2$, so the appropriate k can be read directly off the singular-value spectrum. Inner products are preserved exactly within the subspace, because $\langle V_k^\top x, V_k^\top y\rangle = x^\top P_k y = \langle \hat{x}, y\rangle$; all distortion comes from the part of the signal that lives in the discarded tail.

Two practical caveats. First, the basis V_k must be fit on the full corpus in advance, so out-of-distribution vectors lose alignment and a query's relevant signal may sit partly in the truncated tail. This is why aggressive truncation (rank-8, rank-16) shows the largest distance error in our results. Second, storage is not simply k floats per vector: the shared basis V_k costs dk floats and the coefficients cost nk, so the *measured* compression ratio falls below the nominal d/k. For $k = 64$ the nominal ratio is $4\times$ but amortized storage yields about $3.4\times$.

2.3 Sign (1-bit) quantization and QJL scoring

The most aggressive linear compressor keeps only the sign of each coordinate,

$$\hat{x} = \operatorname{sign}(x) \in \{-1, +1\}^d, \tag{9}$$

collapsing 32 bits per coordinate to 1, a nominal 32× reduction. The geometric content of a sign vector is an angle, not a length: every $\hat{x}$ has fixed norm $\sqrt{d}$, so magnitude is discarded and only direction survives.

Signs encode angles. For a random hyperplane with normal $g \sim \mathcal{N}(0, I_d)$, the probability that two vectors fall on opposite sides is exactly proportional to the angle between them, the Goemans–Williamson identity [4]:

$$\Pr\Big[\operatorname{sign}(g^\top x) \neq \operatorname{sign}(g^\top y)\Big] = \frac{\theta(x,y)}{\pi}, \qquad \theta(x,y) = \arccos \frac{\langle x, y\rangle}{\|x\|\,\|y\|}. \tag{10}$$

Equivalently, sign-quantizing both vectors gives $\mathbb{E}\big[\frac{1}{k}\operatorname{sign}(Rx)^\top \operatorname{sign}(Ry)\big] = 1 - \frac{2}{\pi}\theta(x,y)$, so the dot product of sign codes is a monotone function of the angle and recovers cosine ordering; this is the basis of sign-random-projection hashing (SimHash) [5]. The quantized-JL (QJL) refinement keeps the query in full precision to cut variance. Using the Gaussian identity $\mathbb{E}\,[\,g\,\operatorname{sign}(g^\top x)] = \sqrt{2/\pi}\,x/\|x\|$, the estimator

$$\operatorname{score}(q,x) = \sqrt{\tfrac{\pi}{2}} \cdot \frac{(Rq)^\top \operatorname{sign}(Rx)}{k} \;\approx\; \frac{\langle q, x\rangle}{\|x\|} \tag{11}$$

is unbiased for cosine similarity. Our `sign_1bit` method uses the axis-aligned special case ($R = I$) and restores scale with the stored norm, scoring $\|x\|\,\operatorname{sign}(x)^\top q/\sqrt{d}$.

Where it fails. One bit localizes direction only coarsely, and the estimator's variance is fixed regardless of how close two vectors are. Sign quantization therefore works well when relevant vectors are well spread out on the sphere and fails when the true top-k items cluster tightly, which is exactly the regime where retrieval needs the most angular precision.

2.4 Uniform scalar quantization

Scalar quantization divides each coordinate's range into 2^b equal buckets and rounds to the nearest bucket center. With per-coordinate range $[x_{\min}, x_{\max}]$ the step size is

$$\Delta = \frac{x_{\max} - x_{\min}}{2^b - 1}, \tag{12}$$

and the worst-case rounding error per coordinate is $\Delta/2$, giving the deterministic reconstruction bound

$$\|x - \hat{x}\|^2 \le d \cdot \left(\frac{\Delta}{2}\right)^2. \tag{13}$$

Error scales as 4^{-b}. Modelling the rounding error of each coordinate as uniform on $[-\Delta/2, \Delta/2]$ gives the standard quantization-noise variance [8]

$$\mathbb{E}\,(x_j - \hat{x}_j)^2 = \frac{\Delta^2}{12}, \qquad \mathbb{E}\,\|x - \hat{x}\|^2 = \frac{d\,\Delta^2}{12}. \tag{14}$$

Since $\Delta \propto 2^{-b}$, the mean-squared error falls as 4^{-b}: each extra bit cuts the error by a factor of four (a 6 dB improvement per bit), a clean and predictable quality-versus-bits tradeoff. The compression ratios are exact: $b = 8$ gives $4\times$, $b = 4$ gives $8\times$, and $b = 2$ gives $16\times$. Writing the rounding error as $e = \hat{x} - x$, the scored inner product is $\langle q, \hat{x} \rangle = \langle q, x \rangle + \langle q, e \rangle$, so quantization is harmless precisely when the error e is small and uncorrelated with the query direction.

Why it is a strong baseline. Scalar quantization keeps every coordinate and never mixes them, so it adds no projection noise of the kind JL suffers, only the independent rounding error above. By 4 bits the grid is fine enough that the cosine ranking of neighbors is barely perturbed, and at 8 bits it is effectively lossless. Its weakness appears only at 2 bits, where the grid is coarse and the error e becomes large and correlated across a vector's coordinates, systematically tilting its direction. That directional bias is exactly what TurboQuant's residual stage is built to remove, and as our results show, scalar quantization at 4–8 bits already matches or beats far more elaborate methods.

2.5 TurboQuant: a two-stage pipeline

TurboQuant (Zandieh et al., ICLR 2026) is a *data-oblivious* quantizer that is provably near-optimal for both mean-squared-error (MSE) and inner-product distortion. Its starting observation is that an MSE-optimal quantizer is a *biased* inner-product estimator: minimizing reconstruction error and preserving dot products are different objectives, so a single quantizer cannot serve retrieval well. TurboQuant bridges the gap with a two-stage construction. For a target budget of b bits per coordinate:

- **Stage 0 (rotate).** Each vector is multiplied by a random rotation Π, for instance the Q factor of a random Gaussian matrix. For a vector on the unit sphere this makes every coordinate follow the same scaled Beta distribution, which converges to $\mathcal{N}(0, 1/d)$ in high dimension, and renders distinct coordinates nearly independent so each can be quantized on its own.
- **Stage 1 (MSE-optimal scalar quantization, $b - 1$ bits).** Because the per-coordinate distribution is known in closed form, TurboQuant precomputes the **Max–Lloyd** optimal scalar quantizer [6, 7] for that Beta distribution, the solution of a continuous one-dimensional k-means problem, and applies it to each rotated coordinate. This minimizes the L_2 norm of the residual $r = x - \hat{x}_1$ but, on its own, biases inner products.
- **Stage 2 (1-bit QJL residual).** The residual is captured with the Quantized Johnson–Lindenstrauss transform: store $z = \operatorname{sign}(Sr)$ for a random Gaussian sketch S, one bit per coordinate. Its inverse is an *unbiased* inner-product estimator, so adding it back corrects the Stage-1 bias for one extra bit.

The full estimate uses a full-precision query against both stages,

$$\operatorname{score}(q, x) \;=\; q^\top \hat{x}_1 \;+\; \frac{\sqrt{\pi/2}}{d}\, \|r\|\, (Sq)^\top \operatorname{sign}(Sr), \tag{15}$$

which TurboQuant proves is unbiased, $\mathbb{E}[\operatorname{score}(q, x)] = \langle q, x \rangle$, with near-optimal variance across all bit-widths. The bit accounting matters: a budget of b buys $b - 1$ Stage-1 bits plus the 1-bit residual, so the residual is carved out of the budget rather than added on top.

Why the two stages help. The rotation (Stage 0) destroys any coordinate system in which energy is concentrated on a few axes and gives every coordinate the same known distribution, so a per-coordinate scalar quantizer is both optimal and cheap. Stage 1 then leaves behind a structured error, the same correlated, direction-biasing residual that hurts plain scalar quantization at low bit budgets. The key fact is that this residual, while not negligible, is small in norm and roughly isotropic after rotation, so it is exactly the kind of vector a 1-bit QJL sketch estimates well *without bias*. Spending one bit per coordinate on the residual's inner-product contribution removes most of the Stage-1 bias without storing the residual itself. The payoff is largest where scalar quantization is weakest (2–3 bits) and shrinks once the Stage-1 quantizer is already accurate (4+ bits), where the rotation and residual become overhead.

Our implementation of TurboQuant. We keep the two-stage skeleton but replace each component with a simpler, explicit operation, written out here in full. Let P be a *signed permutation matrix*: a coordinate permutation composed with random ± 1 sign flips. It is orthogonal, $P^\top P = I$, and costs only $O(d)$ to store and apply.

Stage 0 (rotate). $x' = Px$.

Stage 1 (uniform quantize). With per-coordinate range $[\ell_j, h_j]$ measured over the corpus and $L = 2^b - 1$ levels,

$$c_j = \mathrm{round}\left(\frac{x'_j - \ell_j}{h_j - \ell_j} L\right), \qquad \hat{x}'_j = \ell_j + \frac{c_j}{L}(h_j - \ell_j), \tag{16}$$

and we map back to the original frame with $\hat{x}_1 = P^\top \hat{x}'$.

Stage 2 (residual). $r = x - \hat{x}_1$, of which we store only the per-coordinate signs $\mathrm{sign}(r) \in \{\pm 1\}^d$ and the scalar norm $\|r\|$.

The query stays in full precision, and the score is the sum of a stage-1 dot product and a 1-bit residual correction,

$$\mathrm{score}(q, x) = \underbrace{q^\top \hat{x}_1}_{\text{stage-1 reconstruction}} + \underbrace{\frac{\|r\|}{\sqrt{d}}\,\mathrm{sign}(r)^\top q}_{\text{1-bit residual correction}}. \tag{17}$$

Why the residual term works. The correction estimates the true residual contribution $\langle q, r\rangle = \sum_j q_j r_j$ from one bit per coordinate. Writing each coordinate as $r_j = \mathrm{sign}(r_j)\,|r_j|$,

$$\langle q, r\rangle = \sum_j q_j\,\mathrm{sign}(r_j)\,|r_j| \;\approx\; \frac{\|r\|}{\sqrt{d}} \sum_j q_j\,\mathrm{sign}(r_j), \tag{18}$$

where the approximation replaces each magnitude $|r_j|$ by the root-mean-square coordinate magnitude $\|r\|/\sqrt{d}$, using $\sum_j r_j^2 = \|r\|^2$. The substitution is exact when the residual coordinates have equal magnitude and accurate when they are close, which is precisely the near-isotropy the rotation is meant to induce. We therefore keep the residual's dominant scale $\|r\|$ exactly and quantize only its direction. Relative to the published QJL estimator we have set the sketch $S = I$ and dropped the $\sqrt{\pi/2}$ constant; the price is a biased but correctly oriented correction, and the saving is one fewer $O(d^2)$ transform.

Why these simplifications are defensible here. TurboQuant's most intricate machinery targets two conditions our study does not face: adversarial worst-case inputs, and KV-cache attention where the absolute inner-product value feeds a softmax. Our setting is gentler, a fixed corpus of near-isotropic embeddings scored by relative rank, and that gap is what each shortcut trades on.

- **Cheap rotation.** The signed-permutation rotation avoids the $O(d^2)$ cost of a dense one, whose job of spreading concentrated energy is largely already done by the embedding model.
- **Uniform quantization.** It costs a fraction of a bit on near-Gaussian data and buys experimental control: the plain `scalar` baseline uses the identical quantizer, so any measured gain is attributable to the residual stage alone.
- **Residual signs ($S = I$).** This forgoes the unbiasedness guarantee, which matters for absolute-magnitude attention but not for ranked retrieval; our results confirm the stripped-down correction still recovers most of the low-bit benefit (76% versus 63% at matched Stage-1 bits).
- **Labelling by Stage-1 bits.** We add the residual bit on top rather than carving it out; since every figure reports *measured* bits per dimension, this changes no comparison.

The consequence is honesty about bits: our `turboquant_2bit` actually consumes about 3.2 bits per dimension (two Stage-1 bits, one residual bit, plus amortized rotation and norm metadata), not the nominal 2.0.

3 Methods and experimental setup

3.1 Corpus and embeddings

The retrieval corpus is the source text of Professor He Wang's MATH 5110 textbook, *Advanced Linear Algebra for AI*, pulled directly from its public Quarto (`.qmd`) repository at https://github.com/wanghemath/Book-AdvancedLinearAlgebraAI. Chapters are split on section boundaries and then into overlapping character windows (900 characters per chunk, 120-character overlap), yielding **1,380 chunks**. Each chunk is embedded with `text-embedding-3-small` at $d = 256$. Retrieval is ranked by **cosine similarity** throughout (the scoring functions normalize vectors at comparison time), so ranking depends on direction, not magnitude.

3.2 Evaluation queries and metric

We auto-generate **300 stratified queries** by sampling across chunks (using section titles and leading sentences) so that every part of the book is represented, with a fixed seed for reproducibility. For each query, the **full-precision top-k set is the ground truth** (100% by definition). A compressed index is scored by

$$\text{overlap@}k = \frac{\left|\,\text{TopK}_{\text{full}} \cap \text{TopK}_{\text{comp}}\,\right|}{k}. \tag{19}$$

This is recall of the true neighbors, and because it is a set intersection it is rank-blind within the top-k window, which is appropriate when a language model consumes any of the retrieved passages, not only the first. We report $k = 3$ for RAG and $k = 10$ for the token study. We also report **mean relative pairwise distance error** to measure geometry directly, and a **drift breakdown** classifying each query as a perfect match, partial drift, or zero overlap.

Two different geometries. It is worth being explicit that our two quality measures use *different* notions of closeness, and they are not interchangeable. The geometric-distortion metric is built on **Euclidean (L_2) distance**: for each method we compute mean relative pairwise distance error, $\big|\,\|x_i - x_j\| - \|c(x_i) - c(x_j)\|\,\big| / \|x_i - x_j\|$ averaged over pairs, which is exactly the quantity the Johnson–Lindenstrauss lemma bounds. Retrieval, by contrast, ranks neighbors by **cosine similarity**, which depends only on the *angle* between vectors. Because our embeddings are not unit-normalized before compression, these two views do not coincide: L_2 distance mixes angle and magnitude, while cosine discards magnitude entirely. A compressor can therefore shrink Euclidean distance error while still scrambling the angular order that retrieval depends on, which is precisely the JL story below. We deliberately report both so the gap between "preserving geometry" and "preserving rankings" is visible rather than assumed away.

3.3 Implementation

All methods are implemented from scratch in NumPy, sharing one embedding matrix and one random seed. The compression families and the bit/dimension ladders evaluated are: `full_precision`; `jl_{16,32,64,128}`; `rank_{8,16,32,64}`; `sign_1bit`; `scalar_{2,3,4,8}bit`; and `turboquant_{2,3,4,8}bit`. The same scoring functions back a FastAPI + SvelteKit web application that lets a user search the book and compare indexes side by side.

4 Results

4.1 Geometric distortion

Figure 1 reports mean relative pairwise *Euclidean* distance error (lower is better). Reading the bars from left to right, three regimes stand out. The 8-bit quantizers are effectively lossless: `turboquant_8bit` at 0.0002 and `scalar_8bit` at 0.0003 are indistinguishable from full precision at 0.0000. The mid-budget methods form a gentle ramp as bits or dimensions are removed. The largest distortions sit at the right with aggressive rank truncation, where `rank_16` and `rank_8` discard singular directions that carry real signal the inner product depends on. The lesson to carry into the next two figures is that this plot measures *geometry only*: low distance error here will not translate into good retrieval for every method.

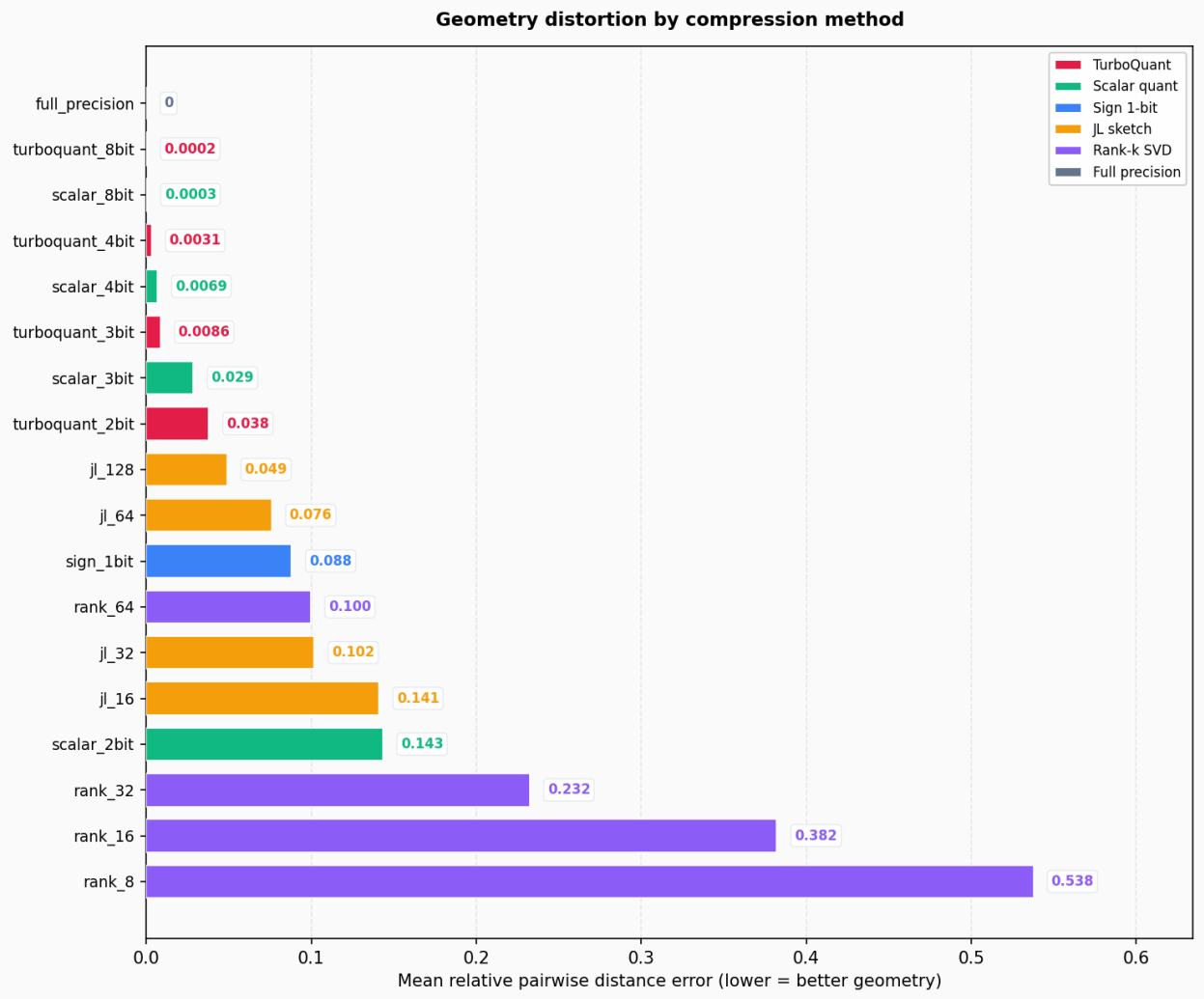

Figure 1: Mean relative pairwise Euclidean distance error by method (lower is better). Eight-bit quantizers are visually indistinguishable from full precision, while aggressive rank truncation (`rank_8`, `rank_16`) produces the largest geometric distortion. Distance error alone does not predict retrieval quality (cf. Figure 2).

4.2 RAG compression frontier

To produce these results we compress the full set of 1,380 chunk embeddings once per configuration, then replay all 300 queries through the scoring function matched to that compressor and average overlap@3 against the full-precision ranking. The compression ratio for each row is computed from the size of the representation actually stored (codes, signs, shared bases, and per-vector norms), not from its nominal label, so the table reflects measured cost rather than design intent. Figure 2 plots top-3 overlap against compression ratio with the Pareto frontier traced in red, and Table 1 lists the underlying numbers.

Table 1: RAG retrieval results (300 queries, $k = 3$, 1,380 chunks).

Method	Top-3 overlap	Bits/dim (measured)	Compression
Full precision	100.0%	32.0	1.0×
JL-128	39.2%	16.0	2.0×
Rank-64	64.8%	9.5	3.4×
Sign 1-bit	47.9%	1.1	28.4×
Scalar 2-bit	62.8%	2.0	16.0×
TurboQuant 2-bit	76.1%	3.2	10.1×
Scalar 3-bit	82.4%	3.0	10.7×
TurboQuant 3-bit	87.0%	4.2	7.7×
Scalar 4-bit	93.2%	4.0	8.0×
TurboQuant 4-bit	93.3%	5.2	6.2×
Scalar 8-bit	99.7%	8.0	4.0×
TurboQuant 8-bit	99.8%	9.2	3.5×

Reading the scatter in Figure 2, each point is one index; better methods push toward the upper-right (high accuracy, high compression), and the red curve connects the Pareto-optimal ones. Two features dominate. First, the JL points (`jl_128`, `jl_64`) sit far *below* the frontier: despite their strong distance guarantees they are the worst retrievers per bit, the clearest visual statement of the geometry-versus-ranking gap. Second, the frontier itself changes hands with the budget. In the low-bit corner the TurboQuant points lie above the scalar points at matched stage-1 bits, where the residual correction is decisive (TurboQuant 2-bit at 76.1% versus scalar 2-bit at 62.8%, a 13-point gain). Once 4 bits are affordable the two families converge in accuracy (93.3% vs 93.2%) and scalar takes over the frontier, because it avoids TurboQuant's rotation and residual overhead and therefore lands at a smaller measured bit cost.

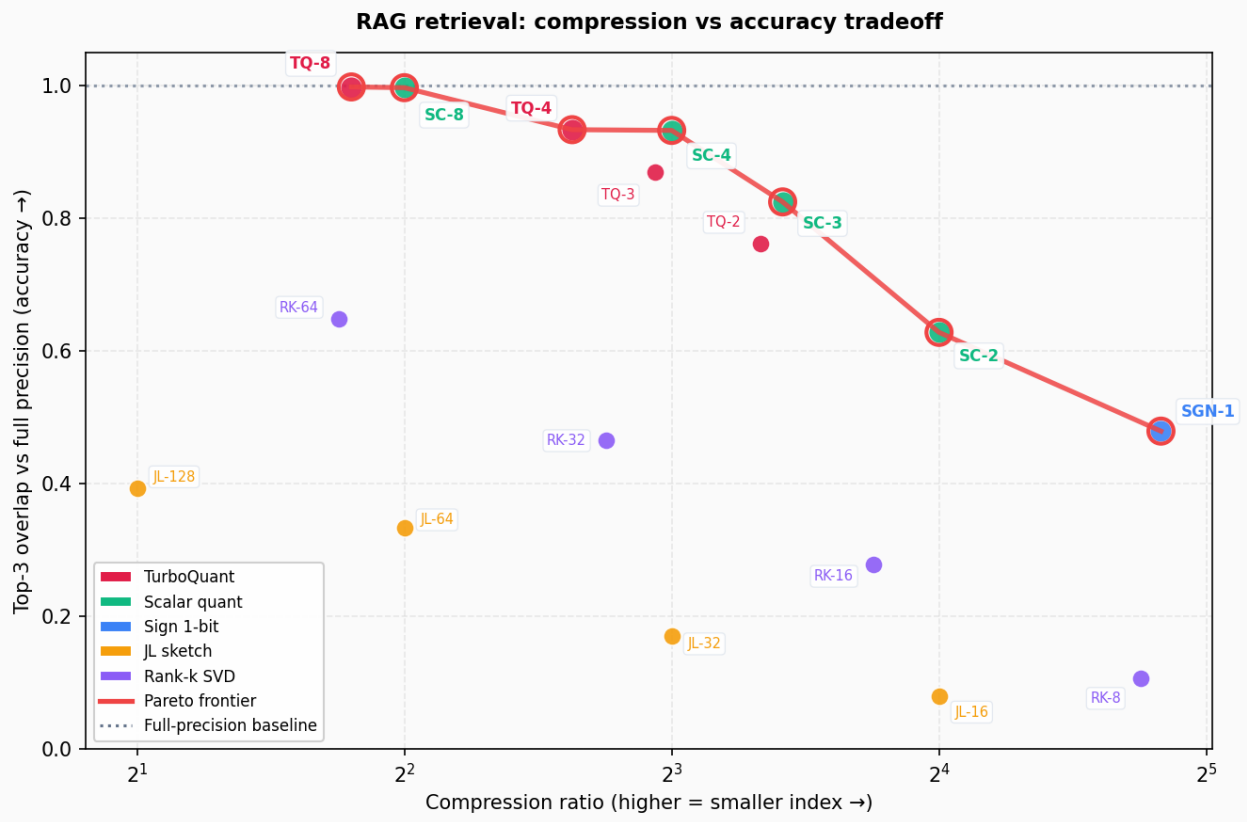

Figure 2: RAG compression frontier: top-3 overlap versus compression ratio (upper-right is better), with the Pareto-optimal methods joined by the red curve. JL configurations sit well below the frontier; TurboQuant owns the low-bit corner, while scalar quantization takes over once 4 bits per dimension are affordable.

4.3 Retrieval drift

Where the frontier reports an average, Figure 3 shows the distribution behind it. Each bar is one method, split into three bands: queries whose top-3 exactly matches full precision (perfect), queries that keep some but not all of the true neighbors (partial drift), and queries that miss the top-3 entirely (zero overlap). The visual story is the shrinking green band. High-bit scalar and TurboQuant configurations are almost entirely perfect matches, whereas `rank_8` and `jl_16` are dominated by partial drift and outright failures. The failures are not spread evenly: drift concentrates on queries where several chunks have nearly equal scores, so a small amount of quantization noise reorders a close top-3. That is exactly the failure mode the QJL residual mitigates at low bit budgets, which is why TurboQuant's perfect band is wider than scalar's at 2–3 bits.

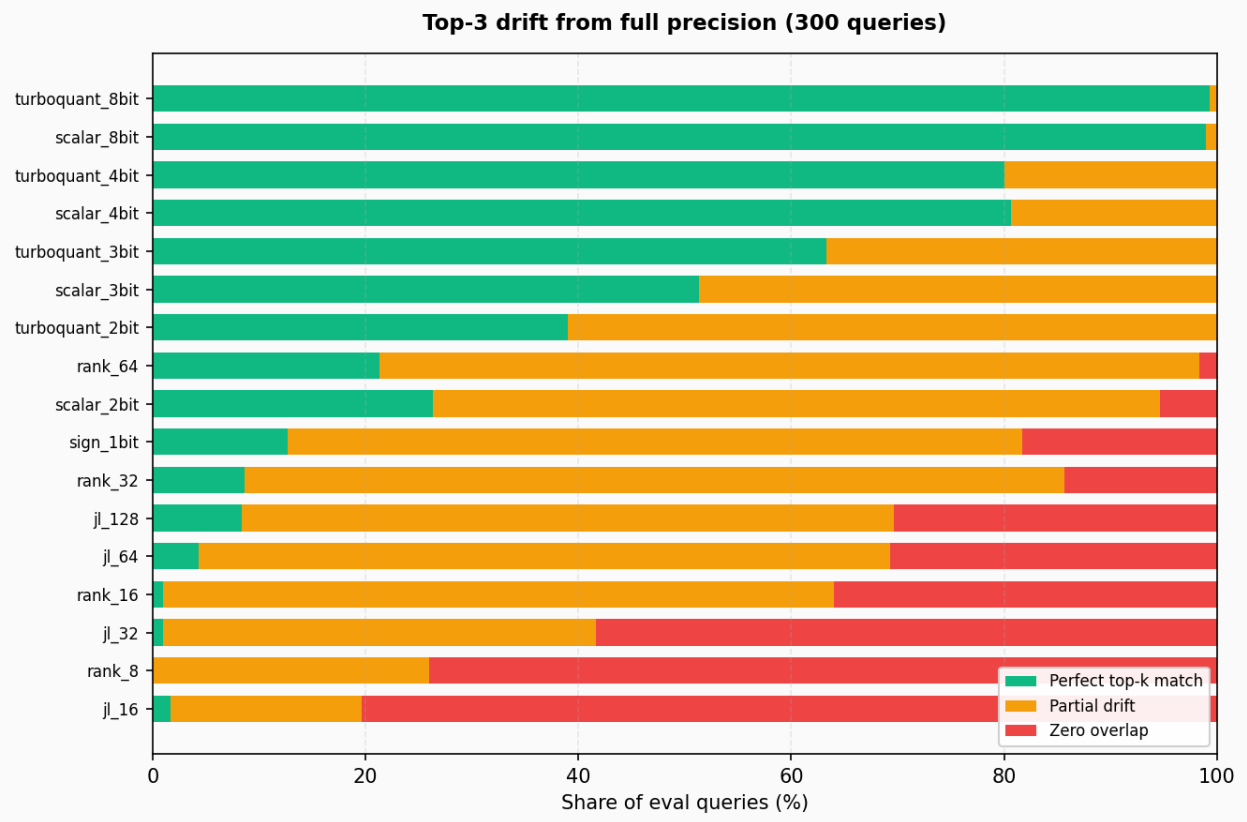

Figure 3: Per-query top-3 drift from full precision across the 300 evaluation queries. Each bar splits into perfect matches, partial drift, and zero-overlap failures; high-bit scalar and TurboQuant keep almost every query in the perfect band, while `rank_8` and `jl_16` are dominated by drift and failures.

4.4 Token study (cross-check)

On a smaller, easier token-embedding task (recall@10), the ordering is the same: TurboQuant leads scalar at matched stage-1 bits (87.0% vs 82.5% at 2 bits; 94.8% vs 90.7% at 3 bits) and the gap closes by 4 bits (97.2% vs 95.8%). This consistency across two corpora supports the conclusions below rather than attributing them to one dataset.

5 Discussion

TurboQuant wins when bits are scarce. When the stage-1 budget is 2–3 bits, scalar quantization is badly biased and the JL residual correction is worth its overhead. TurboQuant 2-bit beats scalar 2-bit by 13 points of top-3 overlap, turning a 1-bit-class method (`sign_1bit` at 47.9%) into a usable retriever (76.1%).

Scalar wins when bits are plentiful. At 4–8 bits the stage-1 quantizer is already accurate, the residual contributes little, and the rotation plus residual metadata are pure overhead. Scalar 4-bit matches TurboQuant 4-bit on accuracy while using fewer total bits, making it the better engineering choice when storage allows.

Distance theory is not retrieval theory. This is the project's sharpest lesson. JL has the strongest theoretical guarantee of any method here, yet `jl_128` retrieves only 39% of the true top-3. The lemma preserves the *magnitude* of pairwise distances, but top-k retrieval depends on their *order*, and a near-isometry can still permute close neighbors. Geometry preservation is necessary but not sufficient for ranking.

Labels are not bit budgets. A method's name encodes a design parameter, not its true cost. TurboQuant 2-bit consumes about 3.2 bits per dimension once residual and metadata are counted, and rank-k stores a shared basis on top of per-vector coefficients. Fair comparison requires plotting against *measured* bits per dimension, which is why every table here reports it.

6 Limitations

This study uses one corpus, one embedding model, and one automatically generated query set; absolute numbers will shift with domain, dimension, and query distribution. The TurboQuant implementation is pedagogical: it uses a lightweight permutation-and-sign rotation rather than a dense random rotation, substitutes uniform scalar quantization for the Beta-distribution Max–Lloyd codebooks, and takes residual signs directly instead of applying the full QJL random projection and its $\sqrt{\pi/2}$ debiasing, so it forfeits the optimality and unbiasedness guarantees of the published method (and its PolarQuant key-cache variant). We evaluate retrieval overlap, not end-to-end generation quality, and we do not reproduce language-model KV-cache inference; the shared object of study is the linear algebra of compressed inner-product search, not the inference stack.

7 Conclusion

Vector compression is governed by a single product, $n \cdot d \cdot$ (bits per dimension), and by a single geometric requirement, approximate preservation of inner products. We surveyed four classical methods and one modern two-stage pipeline, implemented them from scratch, and evaluated them on a realistic textbook RAG task with full-precision ground truth. The results draw a clean line between geometry and ranking: Johnson–Lindenstrauss preserves distances but not retrieval order, sign quantization is extreme but brittle, truncated SVD is optimal in reconstruction yet costly to store, and scalar quantization is a deceptively strong baseline. TurboQuant's quantized-JL residual earns its overhead only at aggressive bit budgets, where it converts a 1-bit-class compressor into a competitive retriever. The practical recommendation is concise: choose TurboQuant when you must compress below 4 bits per dimension, choose scalar quantization otherwise, and always compare methods on measured bits per dimension against a retrieval metric, never on nominal labels or distance error alone.

References

[1] Johnson, W. B., & Lindenstrauss, J. (1984). Extensions of Lipschitz maps into a Hilbert space. *Contemporary Mathematics*, 26, 189–206.

[2] Dasgupta, S., & Gupta, A. (2003). An elementary proof of a theorem of Johnson and Lindenstrauss. *Random Structures & Algorithms*, 22(1), 60–65.

[3] Eckart, C., & Young, G. (1936). The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3), 211–218.

[4] Goemans, M. X., & Williamson, D. P. (1995). Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *Journal of the ACM*, 42(6), 1115–1145.

[5] Charikar, M. S. (2002). Similarity estimation techniques from rounding algorithms. *STOC*, 380–388.

[6] Lloyd, S. P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), 129–137.

[7] Max, J. (1960). Quantizing for minimum distortion. *IRE Transactions on Information Theory*, 6(1), 7–12.

[8] Gray, R. M., & Neuhoff, D. L. (1998). Quantization. *IEEE Transactions on Information Theory*, 44(6), 2325–2383.

[9] Lewis, P., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS*.

[10] Zandieh, A., et al. (2025). Quantized Johnson–Lindenstrauss transforms (QJL). *AAAI*. arXiv:2406.03482.

[11] Zandieh, A., et al. (2026). PolarQuant: Quantizing the KV cache with polar coordinates. *AISTATS*. arXiv:2502.02617.

[12] Zandieh, A., Mirrokni, V., et al. (2026). TurboQuant: Online vector quantization with near-optimal distortion rate. *ICLR*. arXiv:2504.19874.

[13] Google Research (2026). TurboQuant: Redefining AI efficiency with extreme compression.

[14] OpenAI. Embeddings guide: `text-embedding-3-small`, $d = 256$.